Python Socket

import socket

- 소켓 통신 관련 라이브러리 추가

socket.gethostname

- socket.gethostname() 함수
  - 로컬 컴퓨터의 이름 반환
- socket.gethostbyname(host_name) 함수
  - 호스트 이름의 IP 주소 반환
  - 외부 호스트의 IP 주소 얻기
    - remote_ip_address= gethostbyname(외부 호스트의 도메인 이름의 문자열)

- <실행결과>
  - host name: DESKTOP-TQ836S2
  - IP address: 220.67.154.156
  - IP address of www.python.org:146.75.48.223

1_gethostname.py

```
import errno
from ipaddress import ip_address
import socket

host_name = socket.gethostname()
ip_address =socket.gethostbyname(host_name)

print (f"host name: {host_name}")
print (f"IP address: {ip_address}")

remote_host = 'www.python.org'
try:
    remote_ip_address = socket.gethostbyname(remote_host)
    print(f"IP address of {remote_host}:{remote_ip_address}")
except IOError as e:
   print('Reading error: {}'.format(str(e)))

# try: except: 에러 처리
```

IP 주소 포맷 바꾸기

- 문자열 IP 주소를 32비트 바이너리로 변환하기(16진수)
  - socket.inet_aton(ip_addr)
  - Convert an IPv4 address from dotted-quad string format (for example, '123.45.67.89') to 32-bit packed binary format
- 16진수 IP 주소를 문자열로 변화하기
  - socket.inet_ntoa(packed_ip_addr)
  - Convert a 32-bit packed IPv4 address (a bytes-like object four bytes in length) to its standard dotted-quad string representation
- <실행결과>
  - IPAddress: 127.0.0.1 ==> Packed: b'₩x7f₩x00₩x00₩x01', Unpacked: 127.0.0.1
  - IPAddress: 192.168.0.1 ==> Packed: b'₩xc0₩xa8₩x00₩x01', Unpacked: 192.168.0.1

2_IPv4Address.py

```
import socket

for ip_addr in ['127.0.0.1','192.168.0.1']:
    packed_ip_addr = socket.inet_aton(ip_addr)
    unpacked_ip_addr = socket.inet_ntoa(packed_ip_addr)
    print ('IPAddress: {} ==> Packed: {}, Unpacked: {}'.format(
        ip_addr, packed_ip_addr, unpacked_ip_addr))
```

서비스 이름 찾기

- 주어진 포트 번호와 프로토콜 이름으로 서비스 이름 찾기
  - socket.getservbyport(port, protocol_name)
  - socket.getservbyport(53, 'udp')

- 실행결과
  - Port: 80 ==> service name: http
  - Port: 21 ==> service name: ftp
  - Port: 25 ==> service name: smtp
  - Port: 53 ==> service name: domain

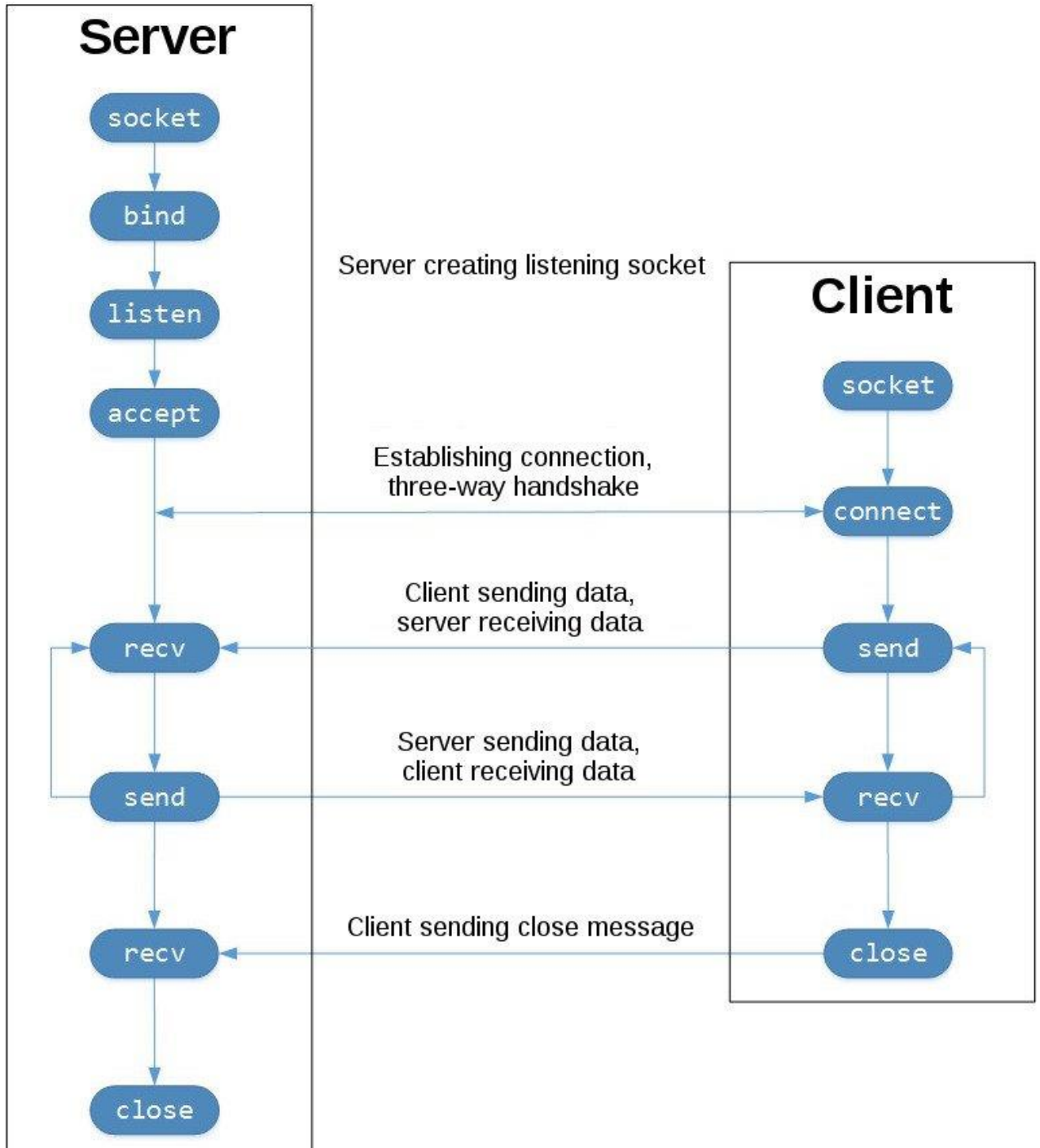
3_service_name.py

```
import socket

protocol_name = 'tcp'
for port in [80, 21, 25] :
    print("Port: {} ==> service name: {}".format(
        port, socket.getservbyport(port, protocol_name)))

print("Port: {} ==> service name: {}".format(
    53, socket.getservbyport(53, 'udp')))
```

TCP 소켓 통신


Server
socket
bind
listen
accept
recv
send
recv
close
Server creating listening socket
Establishing connection,
three-way handshake
Client sending data,
server receiving data
Server sending data,
client receiving data
Client sending close message
Client
socket
connect
send
recv
close

서버 소켓 오브젝트 생성

- s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
  - Socket families
    - socket.AF_INET: IPv4의 인터넷 주소 사용
    - socket.AF_BLUETOOTH: 블루투스 주소 사용
    - socket.AF_INET6: IPv6
  - Socket Types
    - socket.SOCK_STREAM: TCP
    - socket.SOCK_DGRAM: UDP
    - socket.SOCK_RAW: IP

https://realpython.com/python-sockets/#socket-api-overview
https://docs.python.org/3/library/socket.html

소켓 바인드

- socket.bind(address)
  - Address: (hostname, port)
    - Hostname: string ‘www.python.org’ ‘ 201.33.45.67’
      - ‘’== INADDR_ANY, ‘<broadcast>’ ==INADDR_BROADCAST
    - Port: integer
- s.bind((socket.gethostname(), 5000))

연결 요청 대기

- socket.listen([backlog])
  - 서버가 클라이언트의 연결을 받아들일 수 있도록 함
  - backlog: 생략 가능
    - >0
    - 시스템에서 허용가능하지만 아직 받아들여지지 않은 가능한 연결 수를 명시

- s.listen(5)

연결 승낙

- socket.accept()
  - (conn, address) 반환
  - conn: 클라이언트와 통신할 새로운 소켓 객체
    - 클라이언트와 통신할 때 사용하는 주소
  - address: (클라이언트의 IP 주소, port 번호)
- clientsocket, address = s.accept()

명령 프럼프트에서 netstat –an 을 통해 연결 확인 가능

연결 종료

- socket.close()
  - 소켓 종료
  - shutdown() 호출하면 즉시 종료
- socket.shutdown()
  - 소켓을 셧다운시킨다
    - SHUT_RDWR: 양 방향 셧다운
    - SHUT_RD: receive 셧다운
    - SHUT_WR: send 셧다운
- clientsocket.close()

데이터 전송

- socket.send(bytes)
  - socket으로 데이터 전송
- clientsocket.send(bytes("Hi!! there !!!!", "utf-8"))
- bytes(source[,encoding])
  - Source: asci 코드 문자열
  - Ascii 문자열이 아닐 때, 한글 사용 가능
    - source, encoding
    - "Hi!! there !!!!", "utf-8”

4_SimpleServer.py

```
import socket

# 소켓 생성 --> 생성한 소켓은 변수 s로 처리
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind((socket.gethostname(), 5000))
#s.bind(('127.0.0.1', 5000))
s.listen(5)
print(f"wainting")

clientsocket, address = s.accept()
print(f"Connection from {address} has been established")
clientsocket.send(bytes("Hi!! there !!!!", "utf-8"))
clientsocket.close()
```

클라이언트 소켓 생성 및 연결요청

- s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
  - 서버와 통신할 소켓
- socket.connect((hostname, port))
  - Address: (hostname, port)
    - Hostname: string ‘www.python.org’ ‘ 201.33.45.67’
      - ‘’== INADDR_ANY, ‘<broadcast>’ ==INADDR_BROADCAST
    - port: integer
- s.connect((socket.gethostname(), 5000))
  - == s.connect((‘127.0.0.1’, 5000))

데이터 받기

- socket.recv(bufsize)
  - 소켓에서 데이터 받기
  - 받은 바이트 오브젝트 반환
- msg = s.recv(8)
- print(msg.decode("utf-8"))
  - msg.decode(encoding)

4_SimpleClient.py

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((socket.gethostname(), 5000))
msg = s.recv(8)
print(msg.decode("utf-8"))
```

실행 결과

서버

클라이언트

wainting

msg = s.recv(8) 일 때 **Hi!! the**

msg = s.recv(100) 일 때 **Hi!! there !!!!**

Connection from ('220.67.154.156', 63668) has been established

5_echoserver .py

```python
import socket
HOST = ''       # HOST = '127.0.0.1' 동일
PORT = 5000     # 사용하지 않는 임의의 포트
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen(1)
    print('대기')
    conn, addr = s.accept()  # 소켓과 (주소, 포트)로 반환
    with conn:
        print('연결:', addr)
        while True:
            data = conn.recv(1024)
            if not data: break
            conn.sendall(data)
            print('받은 데이터:', data.decode('utf-8'))
        conn.close()
    s.close()
```

5_Echo_client.py

```
import socket
HOST = '127.0.0.1'
PORT = 5000                    # 서버와 같은 포트 사용
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    print('연결 완료')
    s.sendall(bytes('안녕하세요 에코서버','utf-8'))
    print('전송 완료')
    data = s.recv(1024)
    print('받은 데이터:', data.decode('utf-8'))
    s.close()
```

실행 결과

서버

클라이언트

대기

연결 완료

연결: ('127.0.0.1', 58677)

전송 완료

받은 데이터: 안녕하세요 에코서버

받은 데이터: 안녕하세요 에코서버

socketserver class

- *class* `socketserver.`**`TCPServer`**

  (server_address, RequestHandlerClass, bind_and_activate=*True*)
  - internet TCP protocol: 클라이언트와 서버사이에 데이터가 연속적으로 전송
  - *bind_and_activate* is true 라면
    생성자가 자동으로 `server_bind()` 와 `server_activate()`를 자동으로 invoke

```
with socketserver.TCPServer((HOST, PORT), MyTCPHandler) as server:
```

  - `(HOST, PORT):`
  - `MyTCPHandler :data 주고 받는 부분 처리 클래스`
    - BaseRequestHandler 클래스를 서브 클래싱
  - 서버 주소와 요청 처리기 클래스를 전달하여 서버 클래스 중 하나를 인스턴스 화 해야
  - with 문에서 서버를 사용하는 것이 좋음
    - 서버 객체의 handle_request()나 serve_forever() 메서드를 호출하여 하나 이상의 요청을 처리
    - 마지막으로, (with 문을 사용하지 않았다면) server_close()를 호출하여 소켓을 닫음

- **class socketserver.BaseRequestHandler**
  - 모든 socketserver 요청 처리기 객체의 슈퍼 클래스입니다
  - 서브 클래스에서 새 handle() 메서드를 정의해야 하며,
    - 다른 메서드를 재정의할 수 있습니다.
  - 각 요청에 대해 서브 클래스의 새 인스턴스가 만들어집니다.
- setup()
  - 초기화 액션을 수행하기 위해 handle() 메서드 전에 호출됩니다.
  - 기본 구현은 아무것도 수행하지 않습니다.
- handle()
  - 이 함수는 요청을 서비스하는 데 필요한 모든 작업을 수행해야 합니다.
  - 기본 구현은 아무것도 수행하지 않습니다.
  - 몇 가지 인스턴스 어트리뷰트를 사용할 수 있습니다
  - 요청은 self.request로 제공
    - 스트림 서비스의 경우, self.request는 소켓 객체
    - 데이터 그램 서비스의 경우, self.request는 문자열과 소켓 쌍
  - 클라이언트 주소는 self.client_address로 제공
  - 서버 인스턴스는 self.server로 제공
- finish()
  - 필요한 정리 액션을 수행하기 위해 handle() 메서드 이후에 호출됩니다. 기본 구현은 아무것도 수행하지 않습니다. setup()에서 예외가 발생하면, 이 함수가 호출되지 않습니다.

- serve_forever(poll_interval=0.5)
  - 명시적인 shutdown() 요청이 있을 때까지 요청을 처리
  - poll_interval 초마다 shutdown을 확인
  - service_actions()를 호출하는데, 서브 클래스나 믹스인이 주어진 서비스에 특정한 동작을 제공하기 위해 사용할 수 있습니다. 예를 들어, ForkingMixIn 클래스는 service_actions()를 사용하여 좀비 자식 프로세스를 정리합니다.

```
import socketserver

  ~~~~~

if __name__ == "__main__":
    HOST, PORT = "localhost", 9999

     # Create the server, binding to localhost on port 9999
    with socketserver.TCPServer((HOST, PORT), MyTCPHandler) as server:
        # Activate the server; this will keep running until you
        # interrupt the program with Ctrl-C
          server.serve_forever()
```

PYTHON 실행 시 내부 변수 값 할당

__name__ 은 호출 프로세스의 이름 의미
즉, 스크립트 방식이 아닌 형태로 실행 시
if 문 이하 실행

serversocket 클래스 사용 시 추가의 작업
없이도 ctrl+c로 무한 루프 종료 가능

server = **socketserver.TCPServer()** 와
기본적으로 동일하나
server 변수의 사용 블록 지정한 것임

```
class MyTCPHandler(socketserver.BaseRequestHandler):
    """
    The request handler class for our server.
    It is instantiated once per connection to the server, and must
    override the handle() method to implement communication to the
    client.
    """
     def handle(self):
         # self.request is the TCP socket connected to the client
         self.data = self.request.recv(1024).strip()
         print("{} wrote:".format(self.client_address[0]))
         print(self.data)
         # just send back the same data, but upper-cased
         self.request.sendall(self.data.upper())
```